

Transmission vidéo basse latence pour drone FPV

Travail de Bachelor

Non Confidentiel

Départements : TIC

Filière : Informatique et systèmes de communication

Orientation : Systèmes informatiques embarqués

Auteur : Tschantz Nathan

Supervisé par : Favrat Pierre

Date : 11 mai 2026

1 Préambule

Ce travail de Bachelor (ci-après **TB**) est réalisé en fin de cursus d'études, en vue de l'obtention du titre de Bachelor of Science HES-SO en Ingénierie.

En tant que travail académique, son contenu, sans préjuger de sa valeur, n'engage ni la responsabilité de l'auteur, ni celles du jury du travail de Bachelor et de l'École.

Toute utilisation, même partielle, de ce TB doit être faite dans le respect du droit d'auteur.

HEIG-VD
Le Chef du Département

Yverdon-les-Bains, le 11 mai 2026

2 Authentification

Je soussigné, Nathan Tschantz, atteste par la présente avoir réalisé seul ce travail et n'avoir utilisé aucune autre source que celles expressément mentionnées.

Nathan Tschantz
signature image ?

Yverdon-les-Bains, le 11 mai 2026

3 Résumé - français

A écrire a la fin du TB...

Le contexte (très bref), Le problème La méthode Le résultat final

4 Abstract - English

Résumé en anglais

Table des matières

1	Préambule	3
2	Authentification	5
3	Résumé - français	7
4	Abstract - English	7
5	Terminologie et concepts de base	13
	Terminologie et concepts clés	13
6	Introduction	14
6.1	Contexte	14
6.2	Problématique	14
6.3	Objectifs du travail de Bachelor	14
6.4	Structure du document	15
7	Étude de faisabilité	16
7.1	But	16
7.2	Méthodologie d'investigation	16
7.3	Faisabilité côté Émetteur (FreeRTOS / STM32)	16
7.4	Faisabilité côté Récepteur (Linux / OpenWrt)	16
7.5	Conclusion de la faisabilité	17
8	Architecture globale du système	18
8.1	L'émetteur embarqué (TX)	18
8.2	Le relais radio	19
8.3	La station de réception au sol (RX)	19
8.4	Précision de la structure du document	20
9	Configuration de l'environnement de développement embarqué	21
9.1	Intégration du SDK sous Windows	21
9.2	Adaptation de la chaîne de cross compilation	21
9.3	Configuration du débogage matériel (ST-LINK)	22
9.4	Provisionnement et calibration radio (Config Store)	22
9.4.1	Architecture de flashage	22
9.4.2	Résolution du conflit de signature et de version matérielle	22
10	Implémentation du réseau Wi-Fi HaLow	24
10.1	Stratégie de diffusion (UDP Broadcast et No-ACK)	24
10.2	Contournement des contraintes réglementaires (ETSI vs FCC)	24
10.3	Le routage et l'isolation IP (OpenWrt)	25
10.4	Validation de la bande passante vidéo	25

11 Capture et traitement du flux vidéo	26
11.1 Le framework GStreamer et l'optimisation H.264	26
11.2 Pipelines d'émission (TX)	26
11.3 Pipelines de réception et d'affichage (RX)	27
11.4 Identification et contournement de la latence de l'ISP	27
11.5 Outils de diagnostic réseau	27
12 Interfaçage matériel et transfert SPI	28
12.1 Portage de l'application d'émission (De Python à C)	28
12.2 Contrôle de flux matériel (Handshake SPI)	28
13 Itérations matérielles et analyse des performances	29
13.1 Profilage théorique de la chaîne d'émission	29
13.1.1 Temps d'encodage matériel (Jetson Nano)	29
13.1.2 Mesure au cycle d'horloge du traitement MCU (STM32)	29
13.2 Caractérisation physique du transport radio	29
13.3 Isolement de la capture et abandon de la piste OpenHD	30
13.4 Bilan "Glass-to-Glass" et latence résiduelle	30
13.5 Bilan récapitulatif des temps de traitement	31
13.6 Bilan "Glass-to-Glass" et perspectives (Rendu intermédiaire)	31
14 Conclusion intermédiaire	33
14.1 Bilan des réalisations	33
14.2 Objectifs pour la phase finale	33
15 Bibliographie	35
16 Annexes	36
17 Glossaire	37
Index	38

Table des figures

1	structure globale du système	15
2	Architecture globale précisée	18
3	Flux logique du système : (1) Lien réseau Sub-1 GHz, (2) Capture vidéo, (3) Interfaçage SPI, (4) Plateformes de traitement TX et RX.	20
4	Configuration du serveur GDB ST-LINK dans STM32CubeIDE	22
5	Schéma du système de test	30

Liste des tableaux

5 Terminologie et concepts de base

Afin de faciliter la lecture de ce document et la compréhension des enjeux techniques liés à la transmission vidéo basse latence, les concepts et acronymes suivants seront couramment utilisés :

Wi-Fi HaLow (IEEE 802.11ah) : Amendement du standard Wi-Fi conçu pour opérer dans les bandes de fréquences inférieures à 1 GHz (Sub-1 GHz). Contrairement au Wi-Fi classique (2.4 GHz ou 5 GHz), il privilégie la portée (jusqu'à plusieurs kilomètres) et la capacité à traverser les obstacles physiques, au détriment de la bande passante maximale.

FPV (*First Person View*) : "Vol en immersion". Technique de pilotage où la caméra embarquée sur le drone transmet un flux vidéo en temps réel vers les lunettes ou l'écran du pilote, exigeant une latence extrêmement faible et constante.

NLOS (*Non-Line-Of-Sight*) : Situation de communication radio où l'émetteur et le récepteur ne sont pas en ligne de vue directe (présence de bâtiments, de végétation ou de relief). C'est dans ces conditions que les hautes fréquences perdent leur efficacité.

MCS (*Modulation and Coding Scheme*) : Indice (généralement de 0 à 10) définissant le débit physique de la transmission. Il combine un type de modulation radio (ex : BPSK, QAM) et un taux de codage. **Un MCS bas (ex : 1 ou 2)** offre un débit réduit, mais garantit une robustesse maximale face au bruit et aux interférences.

LDPC (*Low-Density Parity-Check*) : Algorithme avancé de correction d'erreurs (FEC). Contrairement au codage convolutif standard (BCC), le LDPC utilise des matrices de parité complexes permettant au récepteur de reconstruire des paquets de données corrompus par de mauvaises conditions radio, évitant ainsi de devoir redemander l'envoi du paquet (ce qui détruirait la latence vidéo).

Couches PHY et MAC : Niveaux fondamentaux du modèle réseau. La **couche PHY** (Physique) gère la transformation des bits en ondes radio. La **couche MAC** (*Media Access Control*) gère les règles d'accès à l'antenne et les retransmissions. Ce projet modifie directement ces couches pour les adapter au flux vidéo.

RTOS (*Real-Time Operating System*) : Système d'exploitation temps réel (comme FreeRTOS, utilisé sur le module de réception). Contrairement à un OS classique (comme Linux), un RTOS garantit l'exécution des tâches dans des délais stricts et déterministes, minimisant ainsi la latence logicielle (gigue) lors de l'affichage de la vidéo.

6 Introduction

6.1 Contexte

Le pilotage de drones FPV (*First Person View*) repose sur un retour vidéo en temps réel. Pour garantir la sécurité du vol et la réactivité du pilote, la fiabilité de la liaison radio et la minimisation de la latence sont absolument critiques. Actuellement, la grande majorité des retours vidéo commerciaux reposent sur des liaisons fonctionnant dans les bandes de fréquences de 2.4 GHz ou 5.8 GHz. Bien que ces hautes fréquences offrent une large bande passante permettant le transfert de vidéos en haute définition, leur capacité de pénétration est en revanche médiocre face aux obstacles physiques.

En environnement urbain, forestier ou industriel (conditions dites *Non-Line-Of-Sight* ou NLOS), la portée de ces systèmes vidéo se trouve drastiquement réduite. À l'inverse, les protocoles opérant dans les bandes inférieures à 1 GHz (Sub-1 GHz, tels que TBS Crossfire ou ExpressLRS) sont devenus le standard industriel pour le lien de contrôle (la radiocommande) grâce à leur robustesse face aux obstacles, mais leur bande passante a toujours été techniquement insuffisante pour y faire transiter de la vidéo.

6.2 Problématique

Pour pallier ce manque et réunir le meilleur des deux mondes, la norme IEEE 802.11ah, également connue sous le nom de Wi-Fi HaLow, se présente comme une alternative prometteuse. Opérant dans la bande de fréquences Sub-1 GHz, elle offre une portée kilométrique et une excellente capacité à traverser les matériaux, tout en proposant des débits de l'ordre du Mégabit par seconde.

Cependant, les standards Wi-Fi classiques n'ont pas été conçus à l'origine pour transporter des flux vidéo continus et déterministes à très faible latence. Les mécanismes natifs d'adaptation de débit (*Rate Control*) et les retransmissions automatiques de paquets visent à garantir l'intégrité absolue des données au détriment du temps de livraison. Pour le vol FPV, l'attente d'une retransmission introduit une gigue (*jitter*) et une latence totalement incompatibles avec les exigences de pilotage.

6.3 Objectifs du travail de Bachelor

L'objectif principal de ce projet est d'étudier, de concevoir et de valider de bout en bout un prototype de transmission vidéo exploitant les avantages physiques du Wi-Fi HaLow. Plutôt que de se limiter à la simple configuration logicielle de modules radio, l'enjeu s'est révélé être une véritable optimisation du système dans son ensemble. Cela implique le contournement des mécanismes d'acquiescement réseau (diffusion unidirectionnelle), le réglage des pipelines de compression vidéo matérielle, ainsi que la mise en place d'une synchronisation physique stricte entre les différents processeurs et microcontrôleurs embarqués pour éviter l'accumulation de données dans les mémoires tampons (*buffers*).

6.4 Structure du document

Ce rapport s'articule autour d'une progression logique, allant de la mise en place des fondations matérielles et logicielles jusqu'à l'optimisation itérative du système complet. Après une étude de faisabilité validant nos choix technologiques, nous détaillerons l'architecture globale retenue.



FIGURE 1 – structure globale du système

Dans un premier temps, nous aborderons la configuration de l'environnement de développement et l'implémentation spécifique du lien réseau Wi-Fi HaLow, éléments qui constituent le socle de notre système.

Une fois ce pont réseau établi, nous décrirons les mécanismes logiciels de capture et de traitement de la vidéo, puis nous détaillerons l'interfaçage matériel (via le bus SPI) permettant d'acheminer ce flux du processeur vers le microcontrôleur d'émission.

Enfin, la dernière partie de ce rapport sera consacrée à l'approche itérative du projet : où seront présentées les différentes combinaisons de plateformes matérielles évaluées (Nvidia Jetson Nano, Raspberry Pi, PC) et où sera exposée l'analyse des mesures de latence ayant permis d'isoler les différents goulots d'étranglement de la chaîne.

7 Étude de faisabilité

7.1 But

Pour réaliser une transmission vidéo robuste face aux obstacles, notre stratégie repose sur le forçage d'un MCS bas et l'utilisation de la technique de correction d'erreurs LDPC. L'objectif de cette étude de faisabilité est de déterminer si ces paramètres, généralement gérés dynamiquement par le matériel, sont configurables manuellement sur les modules Wi-Fi HaLow Morse Micro MM8108 utilisés pour ce projet.

7.2 Méthodologie d'investigation

Dans un premier temps, l'analyse de la documentation constructeur et des interfaces utilisateur (*user-friendly*) d'OpenWrt a révélé que si certains paramètres basiques sont accessibles (comme le choix du canal ou de la largeur de bande), l'activation forcée du LDPC ou le verrouillage strict du MCS ne sont pas exposés par les utilitaires standards (tels que `iw`, `morse_cli` ou `hostapd`). L'investigation s'est donc orientée vers le reverse engineering et l'analyse du code source des pilotes fournis par Morse Micro, tant pour l'environnement FreeRTOS (Émetteur embarqué) que pour l'environnement Linux (Routeur relais).

7.3 Faisabilité côté Émetteur (FreeRTOS / STM32)

L'émetteur (TX), chargé de récupérer la vidéo et d'émettre les ondes radio depuis le drone, repose sur un microcontrôleur STM32 exploitant le système d'exploitation temps réel FreeRTOS. L'investigation du kit de développement logiciel (`mm-iot-sdk`) a permis d'identifier les leviers d'action suivants :

- **Contrôle du MCS** : L'analyse des bibliothèques a mis en évidence la fonction interne `mmwlan_ate_override_rate_control()` issue du fichier d'en-tête `mmwlan.h`. Cette fonction de test (ATE) permet de verrouiller dynamiquement la bande passante et le MCS directement dans le code source de l'application en C.
- **Activation du LDPC** : L'inspection des configurations internes, notamment celles liées au démon `wpa_supplicant` (`config_ssid.h`), indique que la fonctionnalité LDPC est activée par défaut par le pilote lors de la négociation de la connexion.

7.4 Faisabilité côté Récepteur (Linux / OpenWrt)

Le relais radio, configuré en point d'accès (AP) au sol pour réceptionner le flux, repose sur une architecture Linux (OpenWrt). L'analyse du code source du pilote noyau (*kernel driver*) a révélé les éléments suivants :

- **Contrôle du MCS** : Les fichiers sources `rc.c` et `command.c` contiennent des structures permettant de désactiver l'algorithme d'adaptation de débit (`enable_fixed_rate`) et d'imposer un MCS fixe (par exemple, MCS 2). Des tests de compilation croisée (*cross-compilation*) du noyau ont confirmé que ces modifications logicielles peuvent être compilées sous forme de module (`morse.ko`) et injectées à chaud sur le routeur cible.

- **La limite du forçage LDPC** : Initialement, une modification directe des entêtes de paquets MAC a été explorée. L'étude du fichier `skbq.c` laissait supposer la possibilité d'intercepter les trames pour manipuler le registre `morse_ratecode` (en forçant les bits B17 et B18 du champ SIG-1). Cependant, les expérimentations ont démontré qu'il est techniquement impossible d'appliquer et de maintenir ces paramètres matériels bas niveau tant que le routeur n'est pas formellement connecté à un client.

La preuve formelle d'une injection manuelle du LDPC a donc été écartée de l'implémentation finale. Néanmoins, la présence active des paramètres liés au LDPC et au MCS 10 dans le code source du pilote confirme la capacité de la puce à utiliser cette correction d'erreur en interne. Nous partons donc du postulat techniquement fondé que le contrôleur radio l'exploite de manière native.

7.5 Conclusion de la faisabilité

L'étude démontre que l'objectif de manipulation de la couche physique est réalisable sur les points critiques. S'il s'est avéré impraticable de forger manuellement chaque en-tête MAC pour imposer le LDPC en raison de la machine à états du standard Wi-Fi, l'accès au code source du SDK FreeRTOS et des pilotes Linux permet d'appliquer les surcharges (*overrides*) nécessaires pour verrouiller l'utilisation d'un MCS bas. Ces découvertes valident la faisabilité d'une liaison radio déterministe, prérequis absolu pour notre système vidéo.

8 Architecture globale du système

Afin de répondre aux exigences très strictes de la transmission vidéo basse latence pour le pilotage FPV, le système a été divisé en trois entités matérielles distinctes : l'émetteur (destiné à être embarqué sur le drone), le relais radio, et la station de réception au sol. Cette segmentation architecturale offre un double avantage : elle permet de décharger le microcontrôleur Wi-Fi des lourdes tâches d'encodage vidéo, et elle a rendu possible le profilage et l'itération de chaque élément séparément tout au long de la phase de développement.

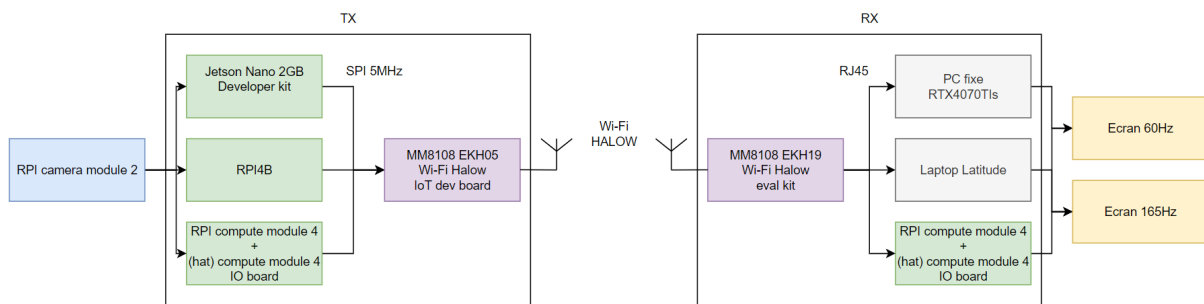


FIGURE 2 – Architecture globale précisée

8.1 L'émetteur embarqué (TX)

L'émetteur embarqué constitue le point de départ de la chaîne d'acquisition. Le module de traitement vidéo n'a pas été figé immédiatement ; plusieurs ordinateurs ont été évalués successivement pour isoler les variables de performance :

- Une **Nvidia Jetson Nano 2GB**, utilisée initialement pour exploiter son encodeur matériel intégré (NVENC) sur le flux brut du capteur CMOS. Le choix de cette plateforme particulièrement puissante visait à éliminer toute inconnue quant aux capacités d'encodage. En établissant une base de référence fiable avec un matériel surdimensionné, il a été possible de valider le pont de communication SPI et la liaison radio sans douter des performances de la source.
- Des **Raspberry Pi 4** (une *Compute Module 4 IO Board* puis une *Raspberry Pi 4B* classique), testées dans un second temps pour évaluer des pipelines de capture vidéo alternatifs dans une démarche de *downscaling*, une fois l'architecture réseau validée. De plus cela a permis d'utiliser les bibliothèques Raspberry natives à la caméra (raspberry camera module 2).

Indépendamment de l'ordinateur utilisé pour la compression (H.264 / H.265), les trames sont ensuite systématiquement transmises via un bus SPI cadencé à 5 MHz vers un microcontrôleur **STM32U5**. Ce MCU agit comme un pont réseau déterministe à ultra-haute vitesse. Il encapsule les données vidéo brutes dans des paquets UDP à l'aide de la pile réseau LwIP, avant de les expédier au module radio **Morse Micro MM8108 (EKH05)** pour la transmission RF sur la bande Sub-1 GHz.

8.2 Le relais radio

Contrairement aux systèmes Wi-Fi traditionnels où le point d'accès traite intensivement les paquets, le relais radio de notre architecture est conçu pour être le plus transparent possible et est resté constant durant tout le projet.

La réception des ondes HaLow est assurée par un routeur **GL.iNet MT3000** équipé d'une carte d'évaluation **EKH19**. Ce routeur, fonctionnant sous le système d'exploitation OpenWrt, est configuré pour agir comme un simple pont logiciel (*Bridge*). Il réceptionne les trames 802.11ah sur son interface sans-fil et les injecte directement sur son interface Ethernet physique, limitant ainsi le temps de traitement logiciel par le CPU du routeur.

8.3 La station de réception au sol (RX)

La station au sol est le dernier maillon de la chaîne, responsable du décodage et de l'affichage. L'objectif de ce projet étant d'observer et de maîtriser chaque source de latence, deux environnements radicalement différents ont été employés :

- **La station de diagnostic (PC Windows)** : Son utilisation a été temporaire et diagnostique. Afin de maintenir un environnement d'analyse logiciel constant, la réception et le décodage ont été réalisés via le framework **GStreamer**. Des tests croisés ont été menés entre un ordinateur portable (Dell Latitude, écran 60 Hz et 165Hz) et une station fixe (NVIDIA RTX 4070 Ti Super, écran 165 Hz). L'unique but de cette plateforme était de prouver que le temps de traitement brut du processeur n'était pas le goulot d'étranglement de l'architecture. Une fois cette hypothèse écartée, l'environnement Windows a été abandonné au profit de Linux. Il est toutefois à noter que sur le plan de la latence, les mesures de latence "Glass-to-Glass" les plus faibles relevées sur cette plateforme ont logiquement été observées en utilisant l'écran 165 Hz.
- **Les stations cibles (Raspberry Pi 4 - CM4 et 4B)** : Le passage définitif sur ces plateformes ARM s'inscrit dans une volonté d'utiliser un système d'exploitation Linux, offrant un meilleur contrôle sur le matériel. L'objectif était de s'affranchir des limitations et de l'overhead de windows. Ces plateformes ont permis d'évaluer et d'implémenter des pipelines logiciels GStreamer également.

8.4 Précision de la structure du document

Comme déterminé précédemment, voici l'ordre dans lequel ce rapport sera structuré de manière plus détaillé.

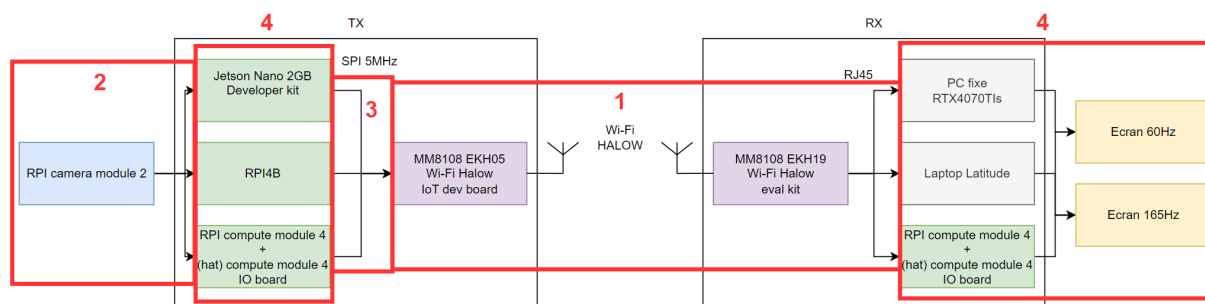


FIGURE 3 – Flux logique du système : (1) Lien réseau Sub-1 GHz, (2) Capture vidéo, (3) Interfaçage SPI, (4) Plateformes de traitement TX et RX.

9 Configuration de l'environnement de développement embarqué

Le développement du firmware pour le microcontrôleur gérant transmission (STM32) repose sur le kit de développement logiciel officiel de Morse Micro (`mm-iot-sdk`). Ce SDK étant nativement conçu pour un environnement Linux avec platformio et des chaînes de cross compilation préinstallées, son intégration sous Windows a nécessité plusieurs adaptations.

9.1 Intégration du SDK sous Windows

Le code source a été récupéré via le dépôt GitHub officiel. L'architecture du projet s'appuyant sur plusieurs dépendances externes vitales (telles que le système d'exploitation temps réel FreeRTOS, la pile réseau LwIP et mbedTLS), une initialisation complète des sous-modules a été requise (`git submodules`).

L'environnement de développement choisi est **STM32CubeIDE**. Le SDK ne respectant pas la structure standard de cet IDE, le projet a été importé en tant que "Makefile Project with Existing Code", en ciblant spécifiquement le répertoire de la carte d'évaluation de l'émetteur (`mm-mm6108-ekh05`, avant sa mise à jour, `mm-mm8108-ekh05` après).

9.2 Adaptation de la chaîne de cross compilation

Sous Windows, les outils de compilation en ligne de commande tels que `make` ne sont pas natifs. Pour pallier ce problème, les variables d'environnement du projet dans STM32CubeIDE ont été modifiées pour inclure les chemins absolus vers les utilitaires internes de l'IDE (`make.exe` et `arm-none-eabi-gcc.exe`).

De plus, le SDK force la recherche de la chaîne de compilation GCC ARM dans les répertoires typiques de Linux (tels que `/opt/`). Il a donc été nécessaire de modifier le fichier de configuration principal `mkcore-arm-cortex-m33f.mk` pour empêcher cette recherche et permettre au système d'utiliser le `PATH` de Windows :

```
# --- MODIFICATION POUR WINDOWS / STM32CubeIDE ---
# On ignore la recherche des dossiers Linux /opt/...
# et on laisse le PATH systeme de STM32CubeIDE trouver le
#   compilateur.
TOOLCHAIN_DIR :=
TOOLCHAIN_BASE := arm-none-eabi-

CC := "$(TOOLCHAIN_BASE)gcc"
CXX := "$(TOOLCHAIN_BASE)g++"
AS := $(CC) -x assembler-with-cpp
OBJCOPY := "$(TOOLCHAIN_BASE)objcopy"
```

9.3 Configuration du débogage matériel (ST-LINK)

Pour permettre l'exécution pas à pas et la pose de breakpoint, le projet a été converti en "Projet ARM" dans l'IDE. La sonde de débogage utilisée est le **ST-LINK** intégré à la carte EKH05, communiquant via le protocole SWD (*Serial Wire Debug*).

FIGURE 4 – Configuration du serveur GDB ST-LINK dans STM32CubeIDE

Il est important de noter une limitation physique de cette architecture : la sonde USB (ST-LINK) ne peut maintenir qu'un seul tunnel de communication actif à la fois. L'utilisation simultanée de la console série (ex : PuTTY, Termit) et du débogueur GDB entraîne des conflits matériels.

9.4 Provisionnement et calibration radio (Config Store)

Une étape indispensable, distincte de la compilation du code C, est le **provisionnement** de la puce Wi-Fi. La puce Morse Micro ne possède pas de mémoire non volatile interne pour ses paramètres vitaux ; elle ne peut pas démarrer sans son micro-code (BCF) et ses paramètres régionaux (*Country Code*), stockés dans une zone mémoire spécifique du STM32 appelée le *Config Store*.

9.4.1 Architecture de flashage

Le processus de flashage repose sur une architecture client-serveur :

1. **Le Serveur (OpenOCD)** : Il établit la connexion matérielle entre le port USB et le processeur ARM via la sonde ST-LINK.
2. **Le Client (Script Python)** : Il lit le fichier de configuration utilisateur (`config.hjson`), génère le binaire correspondant, et commande au serveur de l'écrire dans la mémoire Flash du STM32 (à l'adresse `0x08004000`).

9.4.2 Résolution du conflit de signature et de version matérielle

Lors des premiers déploiements, une erreur bloquait la communication SPI entre le MCU et la puce Wi-Fi (`Transport init failed - Chip ID 0x0000`). Le code embarqué plantait dès l'initialisation du HAL.

L'analyse de ce blocage a révélé un désaccord matériel : le code C et le fichier de configuration cachaient une dépendance stricte à l'ancienne plateforme (MM6108). Notre matériel étant la version MM8108, le binaire de calibration injecté était incorrect. Par une coïncidence extrêmement heureuse, Morse Micro a mis à jour son SDK public la semaine suivante, ajoutant officiellement le support de la cible `mm-mm8108-ekh05`.

Cette mise à jour a permis de rediriger le script vers le fichier de calibration exact de la puce (`bcbf_mf08651_us.mbin`). Combiné à l'ajustement du fichier `config.hjson` pour cibler la région "US", la puce radio a pu démarrer avec succès.

Le module a été sciemment configuré pour cibler la région "US" (États-Unis, norme FCC sur la bande des 915 MHz) au lieu du domaine européen par défaut (norme ETSI sur la bande des 868 MHz). Ce choix est basé par les contraintes de partage du spectre radio. La réglementation européenne impose des règles strictes telles que le *Listen Before Talk* (LBT) et des limites sévères de *Duty Cycle* (temps de transmission maximal). Ces mécanismes forcent le module radio à retenir ou espacer ses paquets pour écouter si le canal est libre.

Lors de nos tests initiaux, ces bridages réglementaires faisaient exploser la latence réseau de base à environ 670 ms. Le passage sur le standard américain permet de désactiver ces restrictions d'émission. Bien que cette configuration soit réservée à un cadre expérimental, elle était indispensable pour ce travail de Bachelor afin de mesurer les performances brutes et réelles de la puce (ramenant la latence radio autour de 3 à 5 ms, tester par ping), indépendamment des limitations légales.

10 Implémentation du réseau Wi-Fi HaLow

La norme IEEE 802.11 intègre intrinsèquement des mécanismes de fiabilité, tels que les accusés de réception (ACK) et les retransmissions automatiques, destinés à garantir l'intégrité des données. Cependant, dans le cadre d'un flux vidéo temps réel pour le FPV, la perte d'une trame est visuellement préférable à l'attente d'une retransmission, qui induit une latence beaucoup trop élevée. L'architecture réseau a donc été configurée pour s'affranchir de ces contraintes.

10.1 Stratégie de diffusion (UDP Broadcast et No-ACK)

Pour contourner l'attente des acquittements matériels de la couche MAC, l'application embarquée sur le STM32 utilise le protocole de transport UDP sur une adresse IP de diffusion *Broadcast*.

Dans le code C exploitant la pile LwIP, l'adresse de destination est configurée sur l'adresse de diffusion locale du sous-réseau :

```
IP4_ADDR(ip_2_ip4(&dest_ip), 192, 168, 12, 255);
```

En ciblant l'adresse .255, la couche réseau LwIP force l'en-tête Wi-Fi à utiliser l'adresse MAC de destination FF:FF:FF:FF:FF:FF. Selon la norme 802.11, les trames diffusées à cette adresse ne déclenchent aucun acquittement (ACK) de la part des récepteurs. L'émetteur expédie ainsi les paquets en flux tendu, ce qui annule la latence induite par les retransmissions au prix de potentielles pertes de paquets.

De plus, la taille de la charge utile (*Payload*) UDP a été fixée à 1400 octets :

```
struct pbuf *p = pbuf_alloc(PBUF_TRANSPORT, 1400, PBUF_REF);
```

Cette valeur approche la taille maximale d'une trame Ethernet standard (MTU), ce qui minimise le ratio d'en-têtes (Overhead MAC+IP+UDP) par rapport aux données utiles.

10.2 Contournement des contraintes réglementaires (ETSI vs FCC)

Lors des premiers essais d'établissement de la liaison radio en MCS 1, une latence de base colossale d'environ 670 ms a été observée sur de simples paquets de test (Ping).

L'analyse de ce délai a mis en évidence l'impact restrictif des normes réglementaires européennes (ETSI) régissant la bande des 868 MHz. Ces normes imposent un mécanisme de partage des ondes strict appelé *Listen Before Talk* (LBT) ainsi que des restrictions de *Duty Cycle* (temps d'émission maximal autorisé). Le module radio retenait intentionnellement les paquets pour respecter ces seuils légaux.

Afin d'évaluer les performances brutes du changement de MCS sans ces bridages logiciels imposés par la région, la configuration de l'émetteur et du récepteur a été basculée sur le domaine réglementaire américain (FCC, bande des 915 MHz). L'utilisation du Canal 27 (915.5 MHz) a permis de désactiver les mécanismes LBT, ramenant immédiatement la latence réseau de base à environ 27 ms pour des paquets de 1450 octets.

10.3 Le routage et l'isolation IP (OpenWrt)

Pour éviter les conflits de routage sur la station de réception, le réseau FPV est strictement isolé sur le sous-réseau 192.168.12.x.

Lors de l'implémentation, un conflit IP majeur bloquait la transmission : le routeur OpenWrt (récepteur) possédait l'adresse 192.168.12.1 simultanément sur son interface Wi-Fi HaLow (**ahwlan**) et sur son interface Ethernet locale (**lan**). Pour résoudre ce conflit, il a suffi de migrer le LAN sur 192.168.13.1.

Concernant le transfert des données à travers le routeur, une première approche a été testée : les paquets UDP entrants sur l'interface sans-fil étaient interceptés (via **tcpdump**) puis redirigés manuellement vers le port Ethernet à l'aide de l'utilitaire **netcat** (**nc**). Cependant, ce traitement en espace utilisateur (*User-Space*) sollicitait lourdement le processeur du routeur et induisait une latence importante.

```
tcpdump -i wlan0 -U -s 0 -w - "udp port 1337" | nc 192.168.12.10 9999
```

Pour optimiser le routage, un pont logiciel (*Bridge*) a finalement été forcé au niveau du système d'exploitation du routeur :

```
brctl addif br-ahwlan wlan0
```

Ce pont matériel permet au flux *Broadcast* en provenance de l'interface sans-fil de traverser le routeur et d'être répliqué directement sur le câble Ethernet (pour autant que la station de réception connectée soit sur le bon sous réseau) sans être intercepté ni traité par la pile TCP/IP ou des utilitaires logiciels, économisant ainsi 10 à 20 ms de délai de traitement additionnel.

10.4 Validation de la bande passante vidéo

La désactivation de l'adaptation automatique de débit (*Rate Control*) a été effectuée dans le SDK via la fonction ATE (Automatic Test Equipment, fonction utilisé en interne pour vérifier le fonctionnement de certaine fonctionnalité durant les phases de tests) :

```
mmwlan_ate_override_rate_control(MMWLAN_MCS_2, MMWLAN_BW_2MHZ, MMWLAN_GI_NONE);
```

En forçant l'utilisation du **MCS 2** sur une largeur de bande de **2 MHz**, la modulation employée (QPSK) garantit une excellente robustesse face aux obstacles physiques. Des mesures empiriques (par capture **tcpdump**) ont validé un débit utile stable d'environ **1.34 Mbps** (120 paquets de 1400 octets par seconde).

Mathématiquement, un flux vidéo brut en résolution 480p (854x480) à 24 images par seconde (codé en 24 bits par pixel) requiert un débit d'environ 236 Mbps. En appliquant une norme de compression vidéo moderne (H.264 / H.265) configurée pour le streaming en temps réel, il est possible de forcer l'encodage vidéo à respecter un certain débit (*Bitrate*). Dans notre cas, les tests montreront par la suite que le taux de compression offrant le meilleur compromis, évitant au maximum les engorgements et les ralentissements, se situe autour de **400 kbps**.

11 Capture et traitement du flux vidéo

Une fois le pont réseau Sub-1 GHz établi, le défi consiste à formater la source vidéo pour qu'elle puisse traverser ce canal étroit sans générer de goulots d'étranglement. Cette section détaille les choix logiciels, l'optimisation des paramètres de compression, et la gestion des tampons matériels.

11.1 Le framework GStreamer et l'optimisation H.264

Pour gérer l'acquisition, la compression et l'encapsulation de la vidéo, le framework *open-source* GStreamer a été retenu de bout en bout. Son architecture modulaire permet de relier directement la sortie d'un capteur matériel à un encodeur, en minimisant les copies en mémoire vive (*Zero-Copy*).

La compression du flux exploite la norme H.264. La configuration a été pensée pour prioriser la vitesse au détriment de l'efficacité de compression absolue. Trois paramètres critiques ont été forcés :

- **Le contrôle de débit strict (CBR - control-rate=1)** : Le débit est verrouillé à 400 kbps pour éviter tout pic de données soudain qui saturerait le bus SPI.
- **La désactivation des images bidirectionnelles (num-B-Frames=0)** : Très important pour la basse latence. L'encodeur n'a plus besoin d'attendre l'image suivante pour calculer l'image courante.
- **L'espacement des images clés (idrinterval=1000)** : L'envoi d'images clés (I-Frames), très volumineuses, est espacé au maximum. Limiter leur fréquence évite les rafales massives de paquets (*bursts*) quiaturent la radio.

11.2 Pipelines d'émission (TX)

Afin de valider cette configuration sur différentes architectures matérielles, les pipelines ont été adaptés :

```
# Sur Nvidia Jetson Nano (NVENC)
gst-launch-1.0 nvarguscamerasrc ! \
'video/x-raw(memory:NVMM),width=640,height=480,framerate=60/1' ! \
nvv4l2h264enc bitrate=400000 control-rate=1 preset-level=1 \
insert-sps-pps=true idrinterval=1000 maxperf-enable=1 num-B-Frames
=0 ! \
h264parse ! fdsink

# Sur Raspberry Pi 4 (V4L2)
gst-launch-1.0 v4l2src device=/dev/video0 ! \
'video/x-raw,width=640,height=480,framerate=60/1' ! \
v4l2h264enc extra-controls="controls,video_bitrate=400000,
video_b_frames=0" ! \
h264parse ! fdsink
```

11.3 Pipelines de réception et d'affichage (RX)

À la réception, le défi s'inverse : il faut décoder et afficher le plus vite possible. L'élément critique est `sync=false`, qui ordonne à GStreamer d'afficher chaque trame dès qu'elle est prête, en ignorant les horodatages.

```
# Sur Station PC Windows (Acceleration D3D11)
gst-launch-1.0 udpsrc port=1337 buffer-size=0 ! \
"video/x-h264,width=640,height=480,stream-format=byte-stream" ! \
h264parse disable-passthrough=true ! d3d11h264dec ! \
queue max-size-buffers=1 leaky=downstream ! \
d3d11videosink sync=false async=false max-lateness=0

# Sur Station cible Raspberry Pi (Bare-Metal via KMS)
gst-launch-1.0 -v udpsrc address=0.0.0.0 port=1337 buffer-size=0 ! \
"video/x-h264,width=640,height=480,framerate=60/1,stream-format=
byte-stream" ! \
h264parse disable-passthrough=true config-interval=-1 ! v4l2h264dec
! \
queue max-size-buffers=1 leaky=downstream ! \
kmssink sync=false max-lateness=0
```

11.4 Identification et contournement de la latence de l'ISP

Malgré ces optimisations, une latence visuelle globale persistait lors des premiers tests. L'analyse différentielle a prouvé que la latence "fantôme" provenait de l'Image Signal Processor (ISP) matériel de la caméra.

Pour effectuer les calculs d'exposition automatique (AE) et de balance des blancs (AWB), l'ISP conserve systématiquement 4 images brutes dans une file d'attente avant de les transmettre à l'encodeur. Mathématiquement, pour un capteur configuré pour échantillonner à 24 images par seconde, la durée d'une image est de 41.6 ms. Le délai induit par les 4 buffers logiciels est donc de :

$$Latence_{ISP} = 4 \times \left(\frac{1000}{24} \right) \approx 166.4 \text{ ms}$$

Pour contourner ce verrouillage matériel massif, la solution implémentée consiste à dissocier la fréquence d'échantillonnage du capteur de la fréquence réseau. En forçant le capteur à capturer à 60 FPS, les 4 buffers de l'ISP sont purgés nettement plus vite ($4 \times 16.6 \text{ ms} = 66.4 \text{ ms}$), réduisant instantanément la latence visuelle de 100 ms.

11.5 Outils de diagnostic réseau

En complément de GStreamer, des utilitaires standards ont été employés pour le diagnostic du flux. `Netcat` (`nc`) a été utilisé pour rediriger manuellement les paquets UDP lors du prototypage, tandis que `tcpdump` a permis de capturer les trames à la volée sur les interfaces sans-fil pour valider la non-fragmentation des paquets HaLow.

12 Interfaçage matériel et transfert SPI

Le transfert du flux vidéo compressé entre le processeur d'acquisition (Jetson / RPi) et le modem radio (STM32) s'effectue via le bus matériel SPI. Cette étape est très importante car elle envoie les données au MCU pour l'émission RF.

12.1 Portage de l'application d'émission (De Python à C)

Initialement développé en Python pour valider le concept, le programme lisant la sortie de GStreamer pour l'injecter sur le bus SPI a été entièrement réécrit en C.

Ce portage répondait à trois impératifs de temps réel :

1. **Élimination de la gestion mémoire asynchrone** : Le *Garbage Collector* de Python induit des micro-pauses aléatoires incompatibles avec la stabilité d'un flux FPV.
2. **Désactivation des caches de l'OS** : En C, la fonction `setvbuf(pipe, NULL, _IONBF, 0)` a été utilisée pour forcer le descripteur de fichier reliant GStreamer au programme à fonctionner en mode "zéro-buffer" (flux tendu).
3. **Précision des horloges** : Remplacement des temporisations logicielles incertaines par des lectures de registres GPIO matériels quasi-instantanées.

12.2 Contrôle de flux matériel (Handshake SPI)

Le transfert s'effectue sur le bus SPI à une fréquence d'horloge de 5 MHz. Mathématiquement, l'envoi d'un paquet UDP contenant une charge utile de 1400 octets (soit 11 200 bits) à cette fréquence requiert un temps de transmission physique incompressible. Ce temps théorique se calcule ainsi :

$$T_{SPI} = \frac{1400 \times 8 \text{ bits}}{5\,000\,000 \text{ bits/s}} = 0.00224 \text{ s} = \mathbf{2.24 \text{ ms}}$$

L'architecture UDP étant configurée sans acquittement, si l'ordinateur enchaîne l'envoi de ces paquets de 2.24 ms trop rapidement, la mémoire RAM du microcontrôleur STM32 déborde (*Buffer Overrun*). Des morceaux d'images sont écrasés avant même d'atteindre le module Wi-Fi, provoquant de sévères artefacts visuels (effets de "bavure").

Pour pallier ce problème, un mécanisme de *Handshake* matériel a été implémenté via une broche de signalisation dédiée (ex : GPIO 78 sur Jetson vers PD15 sur STM32) :

```
// Extrait du controle de flux materiel (Cote Linux)
do {
    lseek(gpio_fd, 0, SEEK_SET);
    read(gpio_fd, &gpio_val, 1);
} while (gpio_val == '0'); // Bloque tant que le STM32 est occupe
```

Le processus est le suivant : le STM32 abaisse ce signal (niveau BAS) dès qu'il reçoit un paquet de 1400 octets via l'interruption SPI. Il ne le relève (niveau HAUT) que lorsque la couche réseau LwIP a fini de transférer le paquet au module Wi-Fi HaLow. Cette synchronisation garantit un flux vidéo propre, dictant la cadence de l'encodeur sans aucune perte interne au système.

13 Itérations matérielles et analyse des performances

Afin d'isoler avec précision les goulots d'étranglement du système et de valider l'architecture proposée, une série de mesures de performances a été menée. L'objectif de cette démarche itérative est de quantifier séparément le temps de transit matériel, le temps de transport réseau, puis le temps de traitement de l'image, afin d'établir un bilan "Glass-to-Glass" complet.

13.1 Profilage théorique de la chaîne d'émission

La première étape a consisté à profiler les temps de traitement incompressibles des composants d'émission.

13.1.1 Temps d'encodage matériel (Jetson Nano)

Pour mesurer le temps de traitement de l'encodeur matériel Nvidia (`nvv4l2h264enc`), les outils de traçage natifs de GStreamer ont été employés via la variable d'environnement `GST_TRACERS="latency(flags=element)"`. L'analyse des journaux d'exécution a démontré que le composant matériel traite et compresse une trame vidéo en un temps extrêmement stable d'environ **1.2 ms**.

13.1.2 Mesure au cycle d'horloge du traitement MCU (STM32)

Pour profiler le transfert SPI et l'encapsulation LwIP, les fonctions classiques (`HAL_GetTick()`) manquaient de résolution. Le code a donc exploité le composant matériel ARM CoreSight DWT (*Data Watchpoint and Trace*). Le registre `DWT->CYCCNT` s'incrémente à chaque coup d'horloge (160 MHz), offrant une résolution de 6.25 nanosecondes.

```
// Demarrage du chronometre DWT dans l'interruption SPI
dwt_start = DWT->CYCCNT;

// ... [Encapsulation pbuf_alloc et envoi udp_sendto] ...

// Arrêt du chronometre et calcul du temps en microsecondes
dwt_end = DWT->CYCCNT;
mcu_cycles = dwt_end - dwt_start;
uint32_t freq_mhz = SystemCoreClock / 1000000; // 160 MHz
uint32_t time_us = mcu_cycles / freq_mhz;
```

L'extraction de cette métrique a révélé un temps de traitement MCU ultra-court de **0.8 ms par paquet**. Le temps total d'émission d'une image (encodage + 2 paquets de 1400 octets transférés à 5 MHz) s'élève théoriquement à :

$$Latence_{TX} = 1.2 + (2 \times (2.24 + 0.8)) = \mathbf{7.28 \text{ ms}}$$

13.2 Caractérisation physique du transport radio

Pour confronter cette théorie à la réalité et isoler la latence de transmission pure, une architecture de test à "déclencheur commun" a été mise en place, s'affranchissant des

problèmes de synchronisation d'horloges réseau.

Deux Raspberry Pi 4B ont été utilisées (TX et RX), avec leurs masses reliées et un bouton poussoir connecté simultanément sur leur broche BCM 17. Lorsqu'on presse le bouton, les deux programmes C (exploitant `mmap` sur `/dev/gpiomem`) détectent le front montant au même instant. Le TX injecte un paquet de test via SPI, et le RX fige son chronomètre haute résolution à la réception du paquet UDP.

Les résultats d'un test en rafale (*Burst*) de 10 paquets consécutifs ont donné un temps total de 124 ms, soit une moyenne en flux de **12.47 ms** par paquet. Ce résultat empirique, très proche de la théorie, prouve que le transport radio Sub-1 GHz est extrêmement rapide.

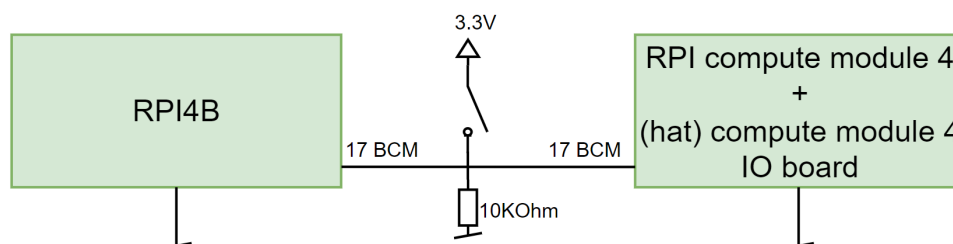


FIGURE 5 – Schéma du système de test

13.3 Isolement de la capture et abandon de la piste OpenHD

L'hypothèse suivante ciblait la pile logicielle de capture vidéo de l'OS. Le déploiement d'un système optimisé *ad hoc* comme OpenHD a été envisagé.

Avant d'engager cette lourde modification, un test de "boucle locale" (*loopback*) a été réalisé sur la Raspberry Pi 4B : capturer le flux caméra, l'encoder et l'afficher directement sur la même carte sans passer par le réseau. La latence mesurée fut de **5 à 7 ms**. Ce résultat a prouvé que la capture et la pile logicielle standard n'étaient pas le goulot d'étranglement. L'itération vers un OS personnalisé a donc été abandonnée.

13.4 Bilan "Glass-to-Glass" et latence résiduelle

La latence "Glass-to-Glass" absolue (entre un événement réel et son affichage sur l'écran final) a été mesurée entre **130 ms et 140 ms**.

L'élimination mathématique de la capture locale (~ 7 ms) et du transport radio (~ 12.5 ms) permet d'isoler un délai inexpliqué de plus de 100 ms. Cette latence résiduelle provient de deux limitations inhérentes au matériel grand public :

1. **L'Image Signal Processor (ISP)** : Comme identifié précédemment, l'ISP conserve 4 images dans une file d'attente pour ses calculs d'exposition, générant jusqu'à 166 ms de délai à 24 FPS. (Contourné partiellement en suréchantillonnant le capteur à 60 FPS).
2. **Le pipeline de réception logiciel** : Le parseur GStreamer (`h264parse`) doit retenir une trame entière pour valider l'intégrité du flux UDP brut. De plus, les gestionnaires de fenêtres des stations de diagnostic (Windows DWM ou X11) imposent

souvent un *Triple Buffering* (synchronisation verticale), ajoutant mécaniquement 50 à 100 ms de latence incompressible avant l’affichage sur les écrans standards.

Le système matériel embarqué et le pont Wi-Fi HaLow ont donc été optimisés à leur limite absolue. La latence restante n’est pas le fait du réseau Sub-1 GHz, mais bien des couches d’abstraction d’acquisition et d’affichage.

13.5 Bilan récapitulatif des temps de traitement

Les différentes itérations matérielles et les mesures de profilage ont permis d’établir une cartographie précise des temps de traitement de chaque maillon de la chaîne, indépendamment des buffers d’affichage.

- **Encodage matériel (Jetson / RPi)** : Les outils de traçage estiment l’encodage H.264 d’une trame à environ **2 ms**.
- **Transfert SPI** : Le transfert d’un paquet de 1400 octets à 5 MHz requiert environ **2.2 ms**.
- **Traitement MCU (STM32)** : Le temps d’encapsulation LwIP, mesuré au cycle d’horloge, est de **0.8 à 1 ms**.
- **Temps de vol radio (Air time)** : Après la désactivation des modes d’économie d’énergie (*Power Saving Mode*) sur le module EKH19, qui induisaient artificiellement une latence de 22 ms, le temps de réponse réseau pur (Ping) entre le PC de réception et le relais s’est stabilisé entre **3 et 5 ms**.
- **Décodage matériel (RX)** : Les temps de décodage sont très similaires d’une plateforme à l’autre : entre **0.5 et 1 ms** sur le GPU du PC portable, et environ **2 à 3 ms** sur la Raspberry Pi 4.

Ce bilan théorique (Encodage + SPI + MCU + Air time + Décodage) donne une somme d’environ **10 à 13 ms**. Ce résultat mathématique corrobore parfaitement la mesure physique de **12.47 ms** obtenue lors du test en rafale avec le déclencheur matériel (bouton partagé entre les deux RPi). De même, le test de boucle locale (capture et affichage sur la même carte) mesuré entre 5 et 7 ms valide notre estimation de 2 ms d’encodage et 3 ms de décodage couplés à un très léger *overhead* de l’OS.

13.6 Bilan "Glass-to-Glass" et perspectives (Rendu intermédiaire)

Malgré la fulgurance démontrée du pont réseau Sub-1 GHz, la latence "Glass-to-Glass" absolue (le délai entre un événement réel et son affichage sur l’écran du pilote) mesurée lors de nos derniers essais se situe entre **130 ms et 140 ms** (sur une architecture combinant RPi 4B et RPi Compute Module).

Ce décalage met en évidence une latence inexpliquée d’environ 120 ms. Dans le cadre de ce rendu intermédiaire, la source exacte de cette latence résiduelle reste à identifier avec certitude.

Plusieurs hypothèses guideront la suite de ce travail de Bachelor :

1. **Goulot d’étranglement sous charge (MCU)** : Une hypothèse supposait qu’un bombardement massif de paquets vidéo ralentissait le STM32 par rapport à un

simple test de ping. Cependant, si le traitement MCU était le goulot d'étranglement, les mémoires tampons déborderaient et la latence s'accumulerait indéfiniment au fil du temps. L'image restant stable, cette piste semble peu probable.

2. **Le pipeline d'acquisition (ISP)** : Comme mentionné précédemment, les files d'attente internes de l'Image Signal Processor de la caméra pour l'exposition automatique pourraient être responsables d'une grande partie de ce délai incompressible avant même que l'image n'atteigne l'encodeur.
3. **L'encapsulation logicielle et l'affichage (RX)** : Bien que le test du bouton ait validé la pile réseau bas niveau du récepteur, l'empilement logiciel GStreamer (**h264parse**) et la synchronisation verticale des écrans de restitution restent des sources potentielles de rétention d'image.

La suite de ce projet se concentrera sur l'isolation stricte de ces dernières variables, afin de cerner l'origine de ces 120 millisecondes et de déterminer les optimisations ultimes possibles pour rapprocher ce système des standards de l'industrie FPV.

14 Conclusion intermédiaire

Le pilotage de drones FPV (*First Person View*) en environnement complexe se heurte depuis toujours à l'incapacité des fréquences standards (2.4 GHz et 5.8 GHz) à pénétrer efficacement les obstacles physiques. Ce travail de Bachelor vise à évaluer une solution de rupture technologique en détournant la norme IEEE 802.11ah (Wi-Fi HaLow), opérant dans la bande Sub-1 GHz, pour y faire transiter un flux vidéo en temps réel.

À ce stade d'avancement du projet, l'architecture fondamentale du système est non seulement établie, mais ses performances ont été démontrées et mesurées de manière rigoureuse.

14.1 Bilan des réalisations

La première grande réussite de ce travail réside dans la maîtrise de la liaison radio HaLow. En contournant les mécanismes d'acquittement de la couche MAC (utilisation de l'UDP Broadcast / mode No-ACK) et en modifiant les configurations de routage, nous avons garanti un canal de transmission fluide et déterministe.

La conception d'un pont de communication matériel (SPI vers Wi-Fi) sur un microcontrôleur STM32, couplé à un mécanisme de *handshake*, a permis de synchroniser le trafic vidéo sans perte. L'approche itérative adoptée a été déterminante : l'utilisation initiale d'une plateforme surpuissante (Nvidia Jetson Nano) a permis de certifier les temps d'encodage via le traçage logiciel GStreamer et d'écarter tout doute sur les performances de compression.

Fort de cette validation de base, le système a pu être efficacement miniaturisé. L'architecture matérielle retenue pour nos tests de bout en bout les plus récents repose ainsi exclusivement sur des plateformes ARM : une Raspberry Pi 4B en émission (TX) et une *Compute Module 4* en réception (RX). L'analyse métrologique de cette chaîne, confirmée par un test de déclencheur physique (bouton partagé), prouve que le transport pur (Encodage matériel, transfert SPI, traitement MCU, temps de vol RF et décodage) s'effectue en **12 à 13 millisecondes**. La technologie Sub-1 GHz n'est donc en rien un frein à la réactivité requise pour le pilotage.

14.2 Objectifs pour la phase finale

Si le lien radio est pleinement validé, la latence "Glass-to-Glass" absolue mesurée actuellement se situe entre **130 ms et 140 ms**. Ce décalage met en évidence une latence résiduelle importante et confirme que les goulots d'étranglement des systèmes vidéo numériques résident désormais dans les couches d'acquisition et de restitution.

Les hypothèses principales pointent vers les files d'attente internes de l'Image Signal Processor (ISP) de la caméra, ainsi que vers les tampons de reconstruction des trames réseau et la synchronisation verticale inhérente aux pilotes d'affichage.

Les semaines restantes avant le rendu final de ce travail de Bachelor seront consacrées à l'isolation et à la caractérisation de ces variables. L'objectif sera de déterminer si des optimisations logicielles de très bas niveau (contournement de l'ISP, affichage *Bare-Metal* strict) permettront de gommer ces 120 ms d'attente, rapprochant ainsi le système des standards de l'industrie FPV et validant définitivement le potentiel du Wi-Fi HaLow pour l'exploration par drone en milieux confinés.

a signé

15 Bibliographie

Références

- [1] Albert Einstein. Zur Elektrodynamik bewegter Körper. (German) [On the electrodynamics of moving bodies]. *Annalen der Physik*, 322(10) :891–921, 1905.
- [2] Morse Micro. *Morse Micro OpenWrt 2.8 Web UI User Guide*. Morse Micro, 2024.
- [3] Morse Micro. *Morse Micro OpenWrt 2.8 Web UI User Guide*. Morse Micro, Paris, 2024.

16 Annexes

17 Glossaire

Index

Couche MAC, 24

DWT, 29

EKH19, 19

FPV, 18

FreeRTOS, 16, 21

GPIO, 28

GStreamer, 19, 26, 29

Jetson Nano, 18

LDPC, 16

LwIP, 18, 21, 29

OpenHD, 30

OpenOCD, 22

OpenWrt, 16, 19

Raspberry Pi, 18

SPI, 18, 29

ST-LINK, 22

STM32, 16, 18

STM32CubeIDE, 21

Tcpdump, 27

Wi-Fi HaLow

MM8108, 16, 18